

Phase 3 & Phase 4 — Plan

Industrial Operations Intelligence Platform — upcoming work (structure + weekly plan).

PHASE 3 — Machine Learning + RAG Document Assistant

Legend: **[NEW]** = added in this phase | **[updated]** = changed in this phase | no marker = already exists

Planned folder & file structure (only the parts that change)

```
ops-intelligence-platform/
|
+-- docker-compose.yml           [updated]
+-- .env.example                 [updated]
|
+-- packages/
|   +-- common/
|   |   +-- ops_common/
|   |   +-- domain/
|   |   +-- registry.py          [updated]
|   |
|   +-- services/
|   |   +-- ml/                  [NEW]
|   |   |   +-- Dockerfile       [NEW]
|   |   |   +-- requirements.txt  [NEW]
|   |   |   +-- app/
|   |   |   |   +-- __init__.py   [NEW]
|   |   |   |   +-- train.py      [NEW]
|   |   |   |   +-- predict.py    [NEW]
|   |   |   |   +-- models/
|   |   |   |   |   +-- predictive_maintenance.py [NEW]
|   |   |   |   |   +-- forecasting.py  [NEW]
|   |   |   |   |   +-- anomaly_detection.py [NEW]
|   |   |   |   +-- registry.py      [NEW]
|   |   |
|   |   +-- rag/                 [NEW]
|   |   |   +-- Dockerfile       [NEW]
|   |   |   +-- requirements.txt  [NEW]
|   |   |   +-- app/
|   |   |   |   +-- __init__.py   [NEW]
|   |   |   |   +-- ingest_docs.py [NEW]
|   |   |   |   +-- chunker.py    [NEW]
|   |   |   |   +-- embeddings.py [NEW]
|   |   |   |   +-- vector_store.py [NEW]
|   |   |   |   +-- retriever.py  [NEW]
|   |   |   |   +-- qa_chain.py   [NEW]
|   |   |
|   |   +-- api-gateway/
|   |   |   +-- requirements.txt  [updated]
|   |   |   +-- api_app/
|   |   |   |   +-- routers/
|   |   |   |   |   +-- v1/
|   |   |   |   |   |   +-- ml.py    [NEW]
|   |   |   |   |   |   +-- rag.py   [NEW]
|   |   |
|   |   +-- dashboard/
|   |   |   +-- app/
|   |   |   |   +-- api_client.py [updated]
|   |   |   |   +-- main.py       [updated]
|   |   |   |   +-- pages/
|   |   |   |   |   +-- predictions.py [NEW]
|   |   |   |   |   +-- documents.py  [NEW]
|   |   |
|   |   +-- data-hub/
|   |   |   +-- postgres/
|   |   |   |   +-- ml_schema.sql  [NEW]
|   |   |   +-- vector/            [NEW]
|   |   |   +-- README.md         [NEW]
|   |
|   +-- services/orchestration/
|   |   +-- dags/
|   |   |   +-- ml_training_dag.py [NEW]
```

What we plan to do in the Phase 3 week

- **Predictive maintenance model.** Use the engineered features from Phase 2 to train a model that predicts which machine (entity) is likely to fail or degrade soon.

- **Forecasting model.** Train a time-series model to forecast future values (for example production or demand) from the daily trend data.
- **Anomaly detection.** Flag unusual readings or sudden changes per entity, so problems are caught early.
- **Model training pipeline.** Add an Airflow DAG that trains and refreshes the models automatically on the stored features.
- **Predictions storage + API.** Save model outputs to a new ML schema and expose them through ML API endpoints.
- **RAG document assistant.** Let users upload documents (manuals, reports, logs); chunk them, create embeddings, and store them in a vector database.
- **Question answering.** Build a retriever + QA chain so users can ask questions like "what does this error code mean?" and get answers from their own documents.
- **Dashboard pages.** Add a Predictions page (model results, risk scores, forecasts) and a Documents page (upload docs and chat with them).
- **Keep it grounded.** RAG answers come only from uploaded documents, with sources shown, so the assistant stays accurate and trustworthy.

PHASE 4 — Agentic AI, Copilot & Production Polish

Legend: **[NEW]** = added in this phase | **[updated]** = changed in this phase | no marker = already exists

Planned folder & file structure (only the parts that change)

```
ops-intelligence-platform/
|
+-- docker-compose.yml           [updated]
+-- .env.example                 [updated]
+-- Makefile                     [updated]
+-- README.md                    [updated]
|
+-- services/
|   +-- agent/                   [NEW]
|       +-- Dockerfile           [NEW]
|       +-- requirements.txt      [NEW]
|       +-- app/
|           +-- __init__.py       [NEW]
|           +-- agent_core.py     [NEW]
|           +-- tools/
|               +-- query_hub.py  [NEW]
|               +-- run_analytics.py [NEW]
|               +-- call_ml.py    [NEW]
|               +-- search_docs.py [NEW]
|           +-- planner.py        [NEW]
|           +-- memory.py         [NEW]
|
|   +-- api-gateway/
|       +-- api_app/
|           +-- routers/
|               +-- v1/
|                   +-- agent.py  [NEW]
|                   +-- auth.py   [NEW]
|
|   +-- dashboard/
|       +-- app/
|           +-- main.py           [updated]
|           +-- pages/
|               +-- copilot.py    [NEW]
|               +-- executive.py  [NEW]
|
+-- packages/
|   +-- common/
|       +-- ops_common/
|           +-- auth.py           [NEW]
|           +-- db.py             [updated]
|
+-- data-hub/
|   +-- postgres/
|       +-- migrations/          [NEW]
|       +-- env.py               [NEW]
|       +-- versions/            [NEW]
|
+-- tests/                       [NEW]
|   +-- test_ingestion.py        [NEW]
|   +-- test_analytics.py        [NEW]
|   +-- test_api.py              [NEW]
|
+-- docs/
|   +-- demo-script.md           [NEW]
|   +-- architecture.md          [NEW]
```

What we plan to do in the Phase 4 week

- **Agentic AI core.** Build an agent that can investigate problems on its own — for example, find the root cause of a drop in production by checking multiple domains.
- **Agent tools.** Give the agent tools to query the hub, run analytics, call the ML models, and search documents, so it can gather evidence and reason step by step.
- **Planner + memory.** Add a planner so the agent breaks a question into steps, and memory so it can carry context across a conversation.
- **AI Copilot page.** A chat interface where managers ask questions in plain language and the agent answers using the platform's data, analytics, ML, and documents.
- **Executive dashboard.** A high-level summary view with key KPIs and alerts for decision-makers.

- **Authentication + roles.** Add proper login (JWT) and role-based access, so different users see what they are allowed to.
- **Proper database migrations.** Wire up Alembic so schema changes are versioned and repeatable (deferred from earlier phases).
- **Tests.** Add automated tests for ingestion, analytics, and the API to make the system reliable.
- **Production polish.** One-command startup, clean logging, a README, and a demo script — so the whole platform runs smoothly and is easy to present.